Problem Statement:

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the oracle docs to learn about comparators.

Input Format

The first line of input contains an integer , representing the total number of students. The next lines contains a list of student information in the following structure:

ID Name CGPA

Constraints

The name contains only lowercase English letters. The contains only integer numbers without leading zeros. The *CGPA* will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each student on a separate line.

Sample Input

5

33 Rumpa 3.68

85 Ashis 3.85

56 Samiha 3.75

19 Samara 3.75

22 Fahim 3.76

Sample Output

Ashis

Fahim

Samara

Samiha

Rumpa

Program:

```
import java.util.*;

// Student class to hold data
class Student {
   private int id;
   private String firstName;
   private double cgpa;

   public Student(int id, String firstName, double cgpa) {
      this.id = id;
      this.firstName = firstName;
      this.cgpa = cgpa;
   }

   public int getId() {
      return id;
   }

   public String getFirstName() {
      return firstName;
   }

   public double getCgpa() {
      return cgpa;
   }
}

public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
```

```
int totalStudents = sc.nextInt();

List<Student> students = new ArrayList<>();

// Read student data
for (int i = 0; i < totalStudents; i++) {
   int id = sc.nextInt();
   String name = sc.next();
   double cgpa = sc.nextDouble();
   students.add(new Student(id, name, cgpa));
}

// Sorting using Comparator
Collections.sort(students, new Comparator<Student>() {
   @Override
   public int compare(Student s1, Student s2) {
      // First: CGPA descending
      if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {
         return Double.compare(s2.getCgpa(), s1.getCgpa());
      }
      // Second: FirstName ascending
      else if (!s1.getFirstName().equals(s2.getFirstName())) {
         return s1.getFirstName().compareTo(s2.getFirstName());
      }
      // Third: ID ascending
      else {
         return Integer.compare(s1.getId(), s2.getId());
      }
   }
});
```

```
        // Print sorted names
        for (Student s : students) {
            System.out.println(s.getFirstName());
        }

        sc.close();
    }
}
```